

Intelligenza Artificiale e Laboratorio

Planning, Sistemi a Regole e Incertezza

Roberto Micalizio

a.a. 2025/2026

Università degli Studi di Torino,
Dipartimento di Informatica



Classical Planning

Cosa è l'Intelligenza Artificiale

*AI is a **science**, not merely technology or engineering. [...] AI is the science of possible forms of intelligence, both individual and collective. [...] AI is the discipline aimed at understanding intelligent beings by constructing intelligent systems.*

Cristiano Castelfranchi

Perché studiare la pianificazione

Prospettiva scientifica

AI punta a comprendere l'intelligenza

- planning è un aspetto fondamentale di un comportamento intelligente (razionale)

Prospettiva tecnologica

AI punta a costruire entità intelligenti

- planning come strumento software per scegliere e organizzare azioni per agenti (sw/hw) autonomi intelligenti

Planning

Planning is the art and practice of thinking before acting.

Patrik Haslum

Automated Planning

Selecting a goal-leading course of action based on a high-level description of the world.

Jörg Hoffmann

Cos'è il Planning

Planning

è un processo deliberativo che sceglie ed organizza le azioni in base all'effetto che ci si aspetta queste producano

AI Planning

è lo studio della calcolabilità di questo processo deliberativo

Modello Concettuale per il Planning

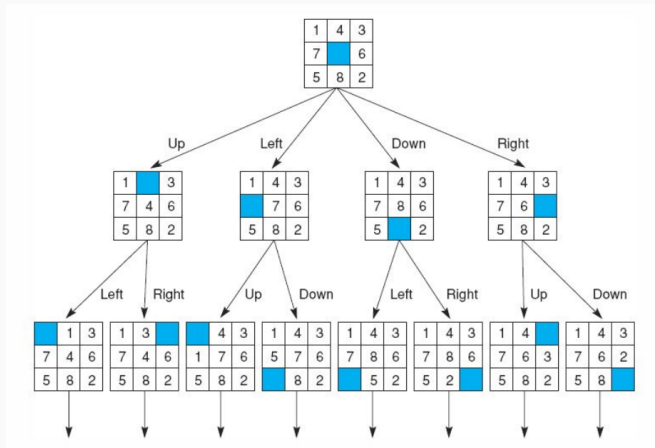
State Transition System (STS)

State Transition System $\Sigma = (S, A, E, \gamma)$

- Dove:
 - $S = \{s_1, s_2, \dots\}$ insieme finito, ricorsivamente enumerabile di stati
 - $A = \{a_1, a_2, \dots\}$ insieme finito, ricorsivamente enumerabile di azioni
 - $E = \{e_1, e_2, \dots\}$ insieme finito, ricorsivamente enumerabile di eventi
 - $\gamma : S \times (A \cup E) \rightarrow 2^S$ è una relazione di transizione di stato:
- Se $a \in A$ e $\gamma(s, a) \neq \emptyset$, allora a è **applicabile** in s
- Applicare a in s causerà una transizione di stato del sistema da s a s' , dove $s' \in \gamma(s, a)$

- Un STS $\Sigma = (S, A, E, \gamma)$ può essere rappresentato come un grafo diretto $G = (N_G, E_G)$ dove:
 - $N_G = S$ è l'insieme dei nodi del grafo coincidente con l'insieme degli stati di Σ
 - E_G è l'insieme degli archi del grafo tale che esiste un arco $s \xrightarrow{u} s'$ (anche rappresentato come $\langle s, u, s' \rangle$) da s a s' etichettato con $u \in A \cup E$, **se e solo se**:
 - $s, s' \in S$ e
 - $s' = \gamma(s, u)$.

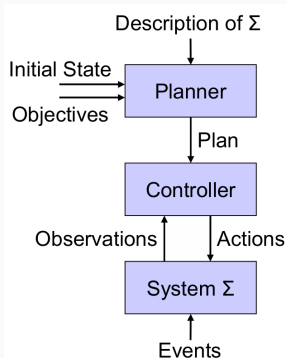
Esempio di STS visto come un grafo



Planning Ingredients: STS, Plans, and Goals

- Un **STS**
 - descrive tutte le possibili evoluzioni di un sistema
- Un **Piano**
 - è una struttura che traccia le azioni necessarie per raggiungere un determinato obiettivo G dato uno stato iniziale I
 - in prima approssimazione è un cammino da I a G nello spazio degli stati tracciato dall'STS
- **Goals** possono essere di diversa tipologia:
 - un goal state s_g o un sottinsieme di possibili goal state in S_g
 - soddisfacimento di condizioni in tutta la sequenza di stati prodotta dalle azioni (es. carburante sempre > 0)
 - ottimizzazione di funzioni di utilità (es. minimizzare costi, tempi, ecc.; massimizzare profitti)
 - vincoli sulle azioni che possono essere eseguite (es. azione a seguita immediatamente, o entro un tempo limite, da azione b)

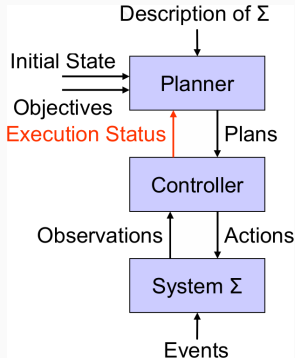
Planning e Plan Execution



- Planner:
 - Data la descrizione di un STS Σ lo stato iniziale, e il goal
 - genera un piano che raggiunge il goal dallo stato iniziale
- Controller:
 - Dato un piano e lo stato corrente (funzione di osservabilità $\eta : S \rightarrow O$)
 - seleziona ed esegue un'azione del piano
- STS Σ :
 - evolve in funzione delle azioni che vengono eseguite e degli eventi che possono accadere

NB. Qui l'assunzione è che gli eventi non interferiscano con le azioni del Controller

Continual Planning



- Challenge: il mondo reale può essere diverso da come è descritto nel modello Σ
- Approccio più realistico: **continual planning** interleaving planning e plan execution
 - plan supervision
 - plan revision (o plan diagnosis)
 - re-planning
- continual planning consente un loop chiuso di feedback tra planner e controller

NB. Qui gli eventi che occorrono nel sistema possono interferire con le azioni pianificate (es. guasti, o condizioni contestuali diverse da quelle ipotizzate nel planning).

Il modello Σ usato per la pianificazione non è sempre in linea con la realtà, è quindi necessario interfogliare pianificazione e azione.

Classical Planning

Dominio Finito

Σ contiene un numero finito di stati

Rilassare A0

- Per:
 - descrivere azioni che producono nuovi oggetti nel mondo
 - trattare fluenti numerici
- Problemi:
 - decidabilità e terminazione del pianificatore

A1: Dominio Completamente Osservabile

Completa Osservabilità

allora la funzione $\eta : S \rightarrow O$ è la funzione identità

Rilassare A1

- Per:
 - trattare stati in cui non tutto è osservabile o può essere conosciuto
- Problemi:
 - poiché in genere si osserva solo un sottoinsieme della realtà, può accadere che $\eta(s) = \eta(s') = o$ con $s \neq s'$
 - le osservazioni sono **ambigue** perché consistenti con più stati possibili
 - determinare lo stato successore può essere problematico
⇒ **belief state**
 - **Conformant planning** (o pianificazione in assenza di osservazioni): pianificare a prescindere dal reale state del mondo, che può non essere noto

A2: Dominio Deterministico

Determinismo

Σ è deterministico, cioè per ogni $s \in S$, $u \in A \cup E$ si ha $|\gamma(s, u)| \leq 1$

Rilassare A2

- Per:
 - pianificare con azioni che possono avere risultati alternativi
- Problemi:
 - il controller deve osservare il risultato reale di ogni azione (problemi di osservabilità A1)
 - il piano soluzione potrebbe contenere dei branch condizionali o iterativi
 - **Conditional Planning**: pianificare sensing actions e piani alternativi in funzione delle possibili risposte dei sensori

Dominio Statico

Σ è statico, ovvero $E = \emptyset$ lo STS può essere ridotto a $\Sigma = (S, A, \gamma)$

Rilassare A3

- Per:
 - modellare domini in cui eventi al di là del controllo dell'esecutore sono possibili
- Problemi:
 - il mondo diventa non deterministico dal punto di vista del pianificatore
 - **Continual Planning**: interfogliare pianificazione ed esecuzione

A4: Goal Semplici

Goal Semplici

- consistono in uno stato s_g da raggiungere o un insieme di stati S_g (è sufficiente che il piano porti ad uno di essi)
- *i goal sono descrizioni parziali di situazioni desiderate*

Rilassare A4

- Per:
 - trattare vincoli su stati e piani, funzioni di utilità/costo, ottimalità
- Problemi:
 - esprimere e ragionare su vincoli ulteriori nella specifica del goal rende il planning computazionalmente costoso

A5: Piani Sequenziali

Piani Sequenziali

Un piano soluzione è una sequenza finita di azioni linearmente ordinate: una sola azione per volta è possibile

Rilassare A5

- Per:
 - sfruttare le capacità degli esecutori nel caso potessero eseguire più azioni
 - non introdurre vincoli che non sono parte del dominio
- Problemi:
 - gestire e ragionare su strutture dati più complesse (es., grafi invece di liste)

A6: Tempo implicito

Tempo Implicito

Le azioni e gli eventi non hanno durata, o in altri termini, hanno tutti durata istantanea

Rilassare A6

- Per:
 - trattare azioni durative (**durative actions**), problemi di concorrenza e deadline
- Problemi:
 - rappresentare e ragionare sul tempo
 - gli effetti delle azioni si sviluppano nel tempo, come si comporta il controller?

A7: Single Agent

Single Agent

Un solo pianificatore e un solo controller (esecutore)

Rilassare A7

- Per:
 - sfruttare meglio le risorse disponibili (più esecutori)
 - trattare situazioni in cui più esecutori sono presenti ma non sono sotto il controllo di un unico pianificatore
- Problemi:
 - **multi-agent planning**: necessità di trattare le interazioni, coordinazione, competizione, negoziazione, game-theoretic planning

Classical Planning Problem

Classical Planning Problem

Un problema di pianificazione classica è $\mathcal{P} = (\Sigma, s_0, S_g)$

- $\Sigma = (S, A, \gamma)$ è il modello del dominio espresso come STS
- $s_0 \in S$ è lo stato iniziale
- $S_g \subset S$ è l'insieme degli stati goal

Una soluzione π ad un problema $\mathcal{P}$

- una sequenza totalmente ordinata di azioni istanziate (**ground**)
 $\pi = \langle a_1, a_2, \dots, a_n \rangle$
- che danno origine ad una sequenza di transizioni di stato
 $\langle s_0, s_1, \dots, s_n \rangle$ tale che:
 - $s_1 = \gamma(s_0, a_1)$
 - $\forall k : 2..n \ s_k = \gamma(s_{k-1}, a_k)$
 - $s_n \in S_g$

Sfide del Classical Planning

- Come rappresentare stati e azioni in modo da non dover esplicitamente enumerare S , A , e γ ?
- Come ricercare una soluzione in modo efficiente? Quali algoritmi? Quali euristiche?
- Come generalizzare le soluzioni? Classical Planning troppo semplice per essere utile nei casi pratici, ma può essere la base per soluzioni in contesti più complessi (rilassando alcune assunzioni)

Complessità della Pianificazione Classica

Perché la Pianificazione è difficile

- E' dimostrabile che il Planning è un task computazionalmente costoso ricorrendo a due problemi decisionali:
 - **PlanSAT**: esiste un piano che risolve un problema di pianificazione?
 - **Bounded PlanSAT**: esiste un piano di lunghezza k ?
- per la pianificazione classica entrambi i problemi sono decidibili (la ricerca avviene in spazio finito)
- ma se estendiamo il linguaggio con simboli di funzione allora:
 - spazio di ricerca diventa infinito
 - **PlanSAT** diventa semidecidibile
 - esiste un algoritmo che termina quando la soluzione esiste
 - potrebbe non terminare quando la soluzione non esiste
 - **Bounded PlanSAT** rimane decidibile

Perché la Pianificazione è difficile

- **PlanSAT** e **Bounded PlanSAT** sono $PSPACE \supset NP$
- In molti casi pratici:
 - **Bounded PlanSAT** è NP -complete
 - **PlanSAT** è P

⇒ trovare una soluzione è meno costoso che trovare una soluzione ottima
- la complessità del planning giustifica la ricerca di euristiche, possibilmente domain-independent, che guidino il pianificatore nella sintesi di una soluzione

Proprietà di un buon algoritmo di pianificazione

Soundness (correttezza)

- Un pianificatore è **corretto** se tutte le soluzioni che trova sono piani corretti, ovvero realmente eseguibili dal controller, questo vuol dire:
 - tutti i goal sono soddisfatti
 - nessuna preconditione di azione è *open* (cioè mancante: o è fornita dallo stato iniziale o da un'altra azione precedente nel piano)
 - nessun vincolo ulteriore è violato (vincoli temporali, istanziazione di variabili, ecc.)

Completeness (completezza)

- Un pianificatore è **completo** se trova una soluzione quando il problema è risolubile
- Un pianificatore è *strettamente completo* se tutte le soluzioni sono mantenute nello spazio di ricerca (i.e., eventuali pruning dello spazio non scartano soluzioni)

Ottimalità

- Un pianificatore è **ottimo** se l'ordine con cui le soluzioni sono trovate è coerente con una qualche misura di qualità dei piani (es. lunghezza, o costo complessivo)